CS-2001
Data Structures
Fall 2023

Stack ADT

Rizwan Ul Haq

Assistant Professor

FAST-NU

rizwan.haq@nu.edu.pk

Today's Lecture

- Introduction to Stack
  - Common applications
  - Array Based-Implementation
  - Linked-list Based Implementation

Stack

- A structure consisting of homogeneous elements and:
  - Insertion and deletions takes place at one end called **top**
  - It is a commonly used abstract data type with two major operations, namely **push** and **pop**

- Other names
  - Last In First Out **(LIFO)** Structure
  - First In Last Out **(FILO)** Structure

Real Life examples

- Books on floor
- Stack of chairs
- In programming, consider doing **X = (A+B) * (C+D)**



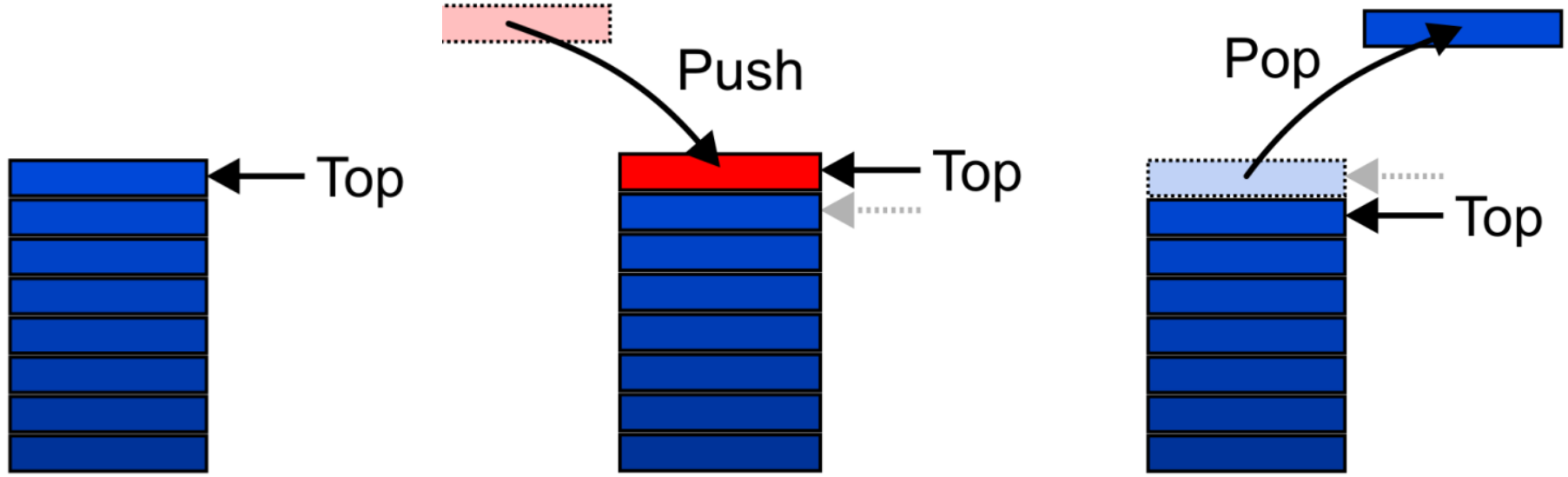


Stack ADT Operations

- Stack ADT emphasizes specific operations
  - Uses an explicit linear ordering (Random access is intentionally revoked)
  - Insertions and removals are performed individually
  - Inserted objects are pushed onto the stack
  - **Top** of the stack is the most recent object pushed onto the stack
  - **Push** and **Pop** operations changes the current top value of the stack

Stack ADT – Operations

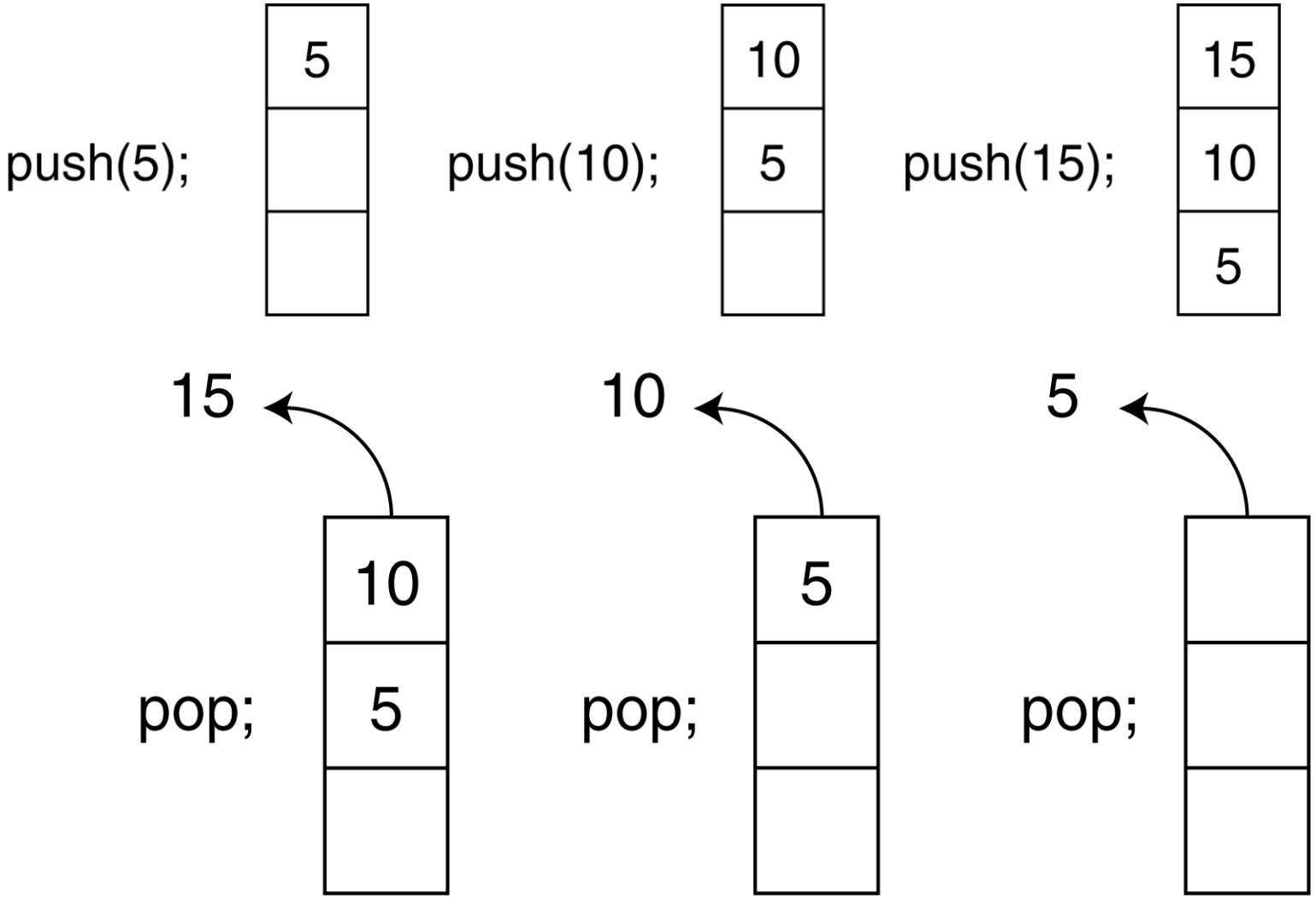
- Graphically, the stack operations are viewed as follows:



Stack ADT – Operations

- CreateStack(S)
  - Make Stack S be an empty stack
- Top(S)
  - Return the element at the top of stack S
- Pop(S)
  - Remove the top element of the stack
- Push(S, x)
  - Insert the element x at the top of the stack
- Empty(S)
  - Return true if S is an empty stack and return false otherwise

Push and Pop operations of Stack



Applications

- Many applications
  - Parsing code
    - Matching parenthesis problem
  - Tracking function calls (Call stack)
  - Reversing a string
  - Infix to postfix Conversion
  - Backtracking in Depth-First-Search

- The stack is a very simple data structure
  - Given any problem, if it is possible to use a stack, this significantly simplifies the solution

Use of Stack in Function Calls

- When a function begins execution an **activation record** is created to store the current execution environment for that function

- Activation records all the necessary information about a function call, including
  - Arguments passed by the caller function
  - Local variables
  - Content of the registers
  - Return address to the caller function
    - Address of instruction following the function call

Use of Stack in Function Calls

- **Each invocation** of a function has its own **activation record**
- **Recursive/Multiple** calls to the functions require **several activation** records to exist simultaneously
- A **function returns** only after **all functions it calls have returned** Last In First Out (LIFO) behavior
- A program/OS keeps track of all the functions that have been called using **run-time stack** or **call stack**

Runtime Stack Example

```
void main() {
        int a = 3;
        f1(a); // statement A
        cout << endl;
}

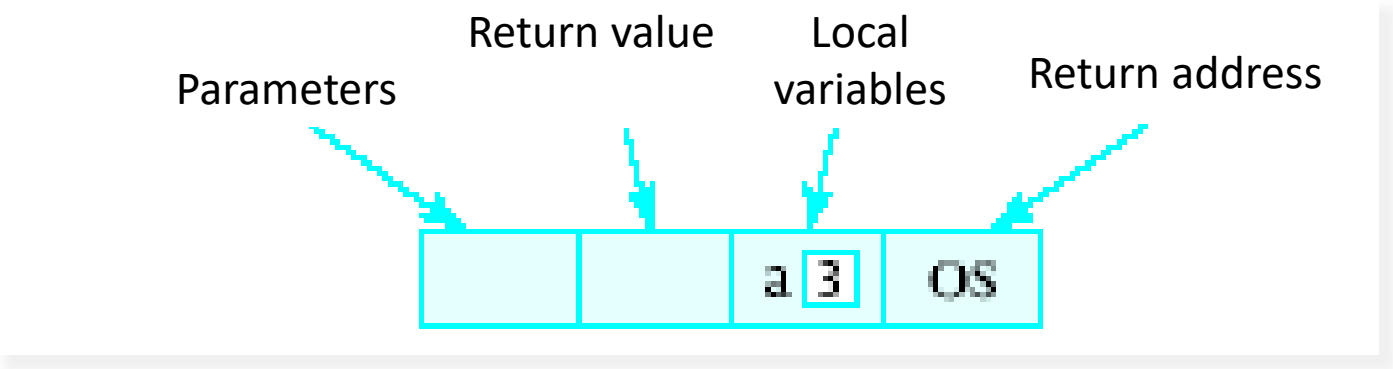
void f1(int x) {
        cout << f2(x + 1); // statement B
}

int f2(int p) {
        int q = f3(p / 2); // statement C
        return 2 * q;
}

int f3(int n) {
        return n*n + 1;
}
```

Runtime Stack

- When a function is called …
  - Copy of activation record pushed onto run-time stack
  - Arguments copied into parameter spaces
  - Control is transferred to starting address of body of function



OS denotes that when execution of main() is completed, it returns to the operating system

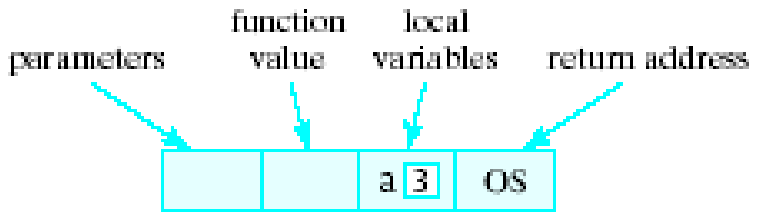
Runtime Stack Example

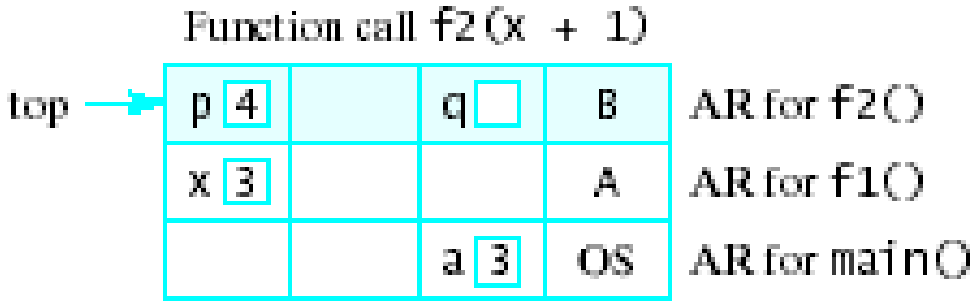
```
void main() {
        int a = 3;
        f1(a); // statement A
        cout << endl;
}

void f1(int x) {
        cout << f2(x + 1); // statement B
}

int f2(int p) {
        int q = f3(p / 2); // statement C
        return 2 * q;
}

int f3(int n) {
        return n*n + 1;
}
```





Static and Dynamic Stacks

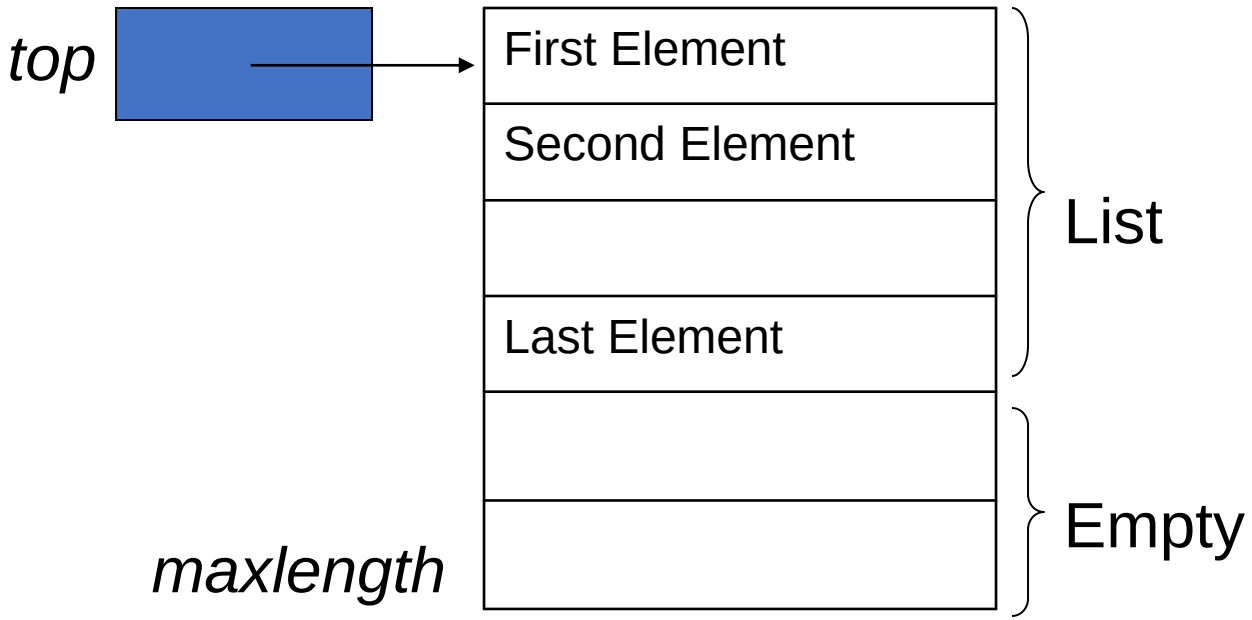
- There are two kinds of stack data structure -
  - a) **static**, i.e. they have a **fixed size**, and are *implemented as* **arrays**.
  - b) **dynamic**, i.e. they **grow in size** as needed, and *implemented as* **linked lists**

Implementation of Stack

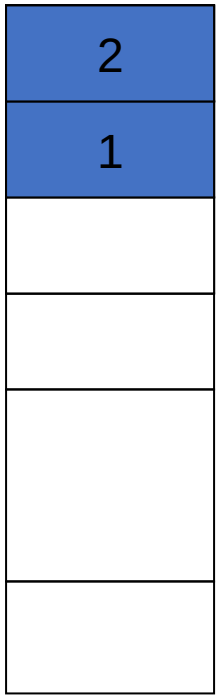
Array Based

Array Implementation of Stack (ver. 1)

- First Implementation
  - Elements are stored in contiguous cells of an array
  - New elements can be inserted to the top of the list



Array Implementation of Stack (ver. 1)



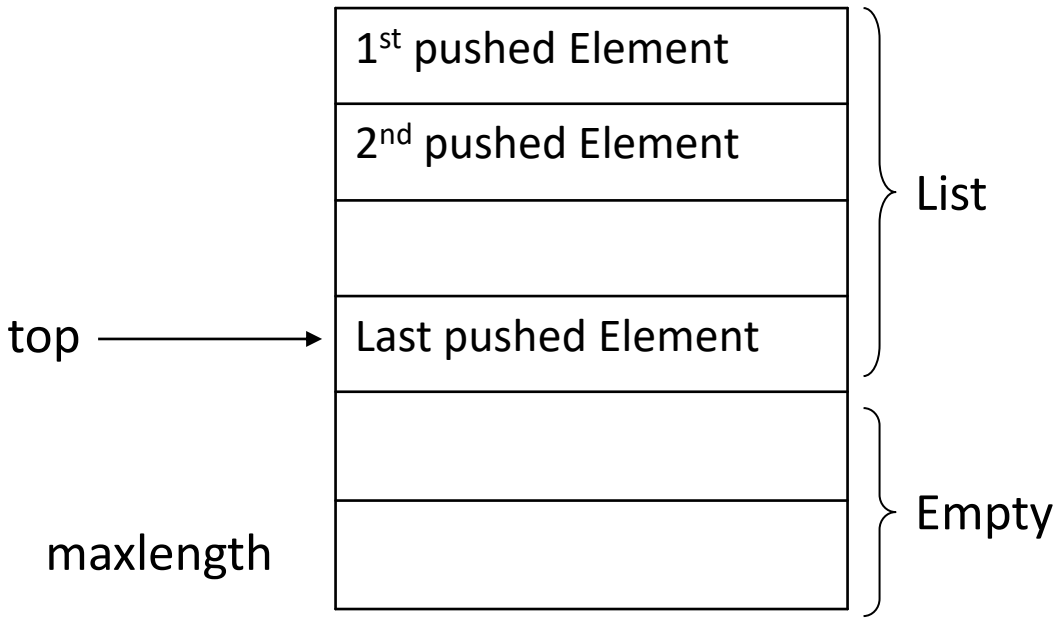
- Problem with this implementation
  - Every PUSH and POP requires moving the entire array up and down

Array Implementation of Stack (ver. 2)

- Since, in a stack the insertion and deletion take place only at the top, so…
- **A better Implementation:**
  - Anchor the bottom of the stack at the start (bottom) of the array
  - Let the stack grow towards the end (top) of the array
  - **Top** indicates the current position of the first stack element

Array Implementation of Stack (ver. 2)

A better Implementation:



Array Implementation

```
class IntStack {
private:
      int *stackArray;
      int stackSize;
      int top;
public:
      IntStack(int);
      ~IntStack();
      bool push(int);
      bool pop(int &);
      bool isFull();
      bool isEmpty();
};
```

Array Implementation

```
//Constructor
IntStack::IntStack(int size)
{
        stackArray = new int[size];
        stackSize = size;
        top = -1;
}

//Destructor
IntStack::~IntStack(void)
{
        delete[] stackArray;
}
```

Array Implementation

```
//isFull function
bool IntStack::isFull(void) {
	return (top == stackSize-1);
}

//isEmpty function
bool IntStack::isEmpty(void)
{
	return (top == -1);
}
```

Array Implementation

- Member function **push** pushes the argument onto the stack

```
bool IntStack::push(int num)
{
	if (isFull())
	{
		cout << "The stack is full.\n" ;
		return false;
	}
	else
	{
		top++;
		stackArray[top] = num;
	}
	return true;
}
```

Array Implementation

- **Pop** function removes the value from top of the stack and returns it as a reference

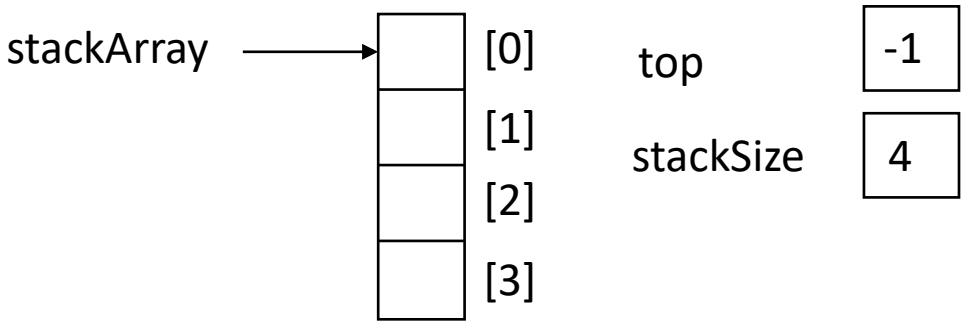
```
bool IntStack::pop(int &num)
{
	if (isEmpty())
	{
		cout << "The stack is empty.\n";
		return false;
	}
	num = stackArray[top];
	top--;
	return true;
}
```

Using Stack

```
int main() {
        IntStack stack(4);




}
```



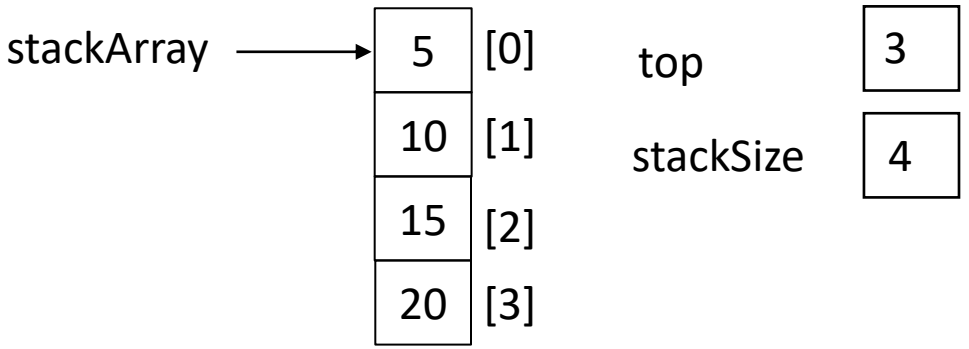
Using Stack

```
int main() {
        IntStack stack(4);

        int catchVar;
        cout << "Pushing Integers\n";
        stack.push(5);
        stack.push(10);
        stack.push(15);
        stack.push(20);




}
```



Using Stack

```
int main() {
        IntStack stack(4);

        int catchVar;
        cout << "Pushing Integers\n";
        stack.push(5);
        stack.push(10);
        stack.push(15);
        stack.push(20);

        cout << "Popping...\n";
        stack.pop(catchVar);
        cout << catchVar << endl;
        stack.pop(catchVar);
        cout << catchVar << endl;
        stack.pop(catchVar);
        cout << catchVar << endl;
        stack.pop(catchVar);
        cout << catchVar << endl;
}
```

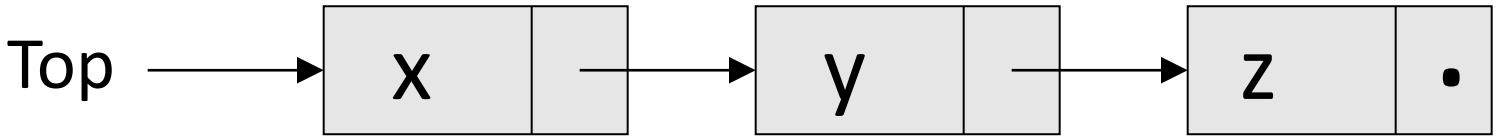
Output:
Pushing Integers
Popping…
20
15
10
5

Implementation of Stack

Pointer Based

Pointer-based Implementation of Stacks

- Stack can expand or shrink with each **push** or **pop** operation
- **Push** and **pop operate** only on the **head node**, i.e., the first node of the list



Pointer-based Implementation of Stacks

```
class node
{
public:
        int data; node *next;
};
class Stack
{
        node *top;
public:
        Stack();
        ~Stack();
        void Push(int newelement);
        bool Pop(int &);
        bool IsEmpty();
        void makeNull();
};
```

Pointer-based Implementation of Stacks

```
//Constructor
Stack::Stack()
{
        top = NULL;
}


//IsEmpty function returns true if the stack is empty
bool Stack::IsEmpty()
{
        return (top == NULL);
}
```

Pointer-based Implementation of Stacks

- **Push** function inserts a node at the top/head of the stack

```
void Stack::Push(int newElement)
{
        node *newptr = new node;
        newptr->data = newElement;

        newptr->next = top;
        top = newptr;
}
```

Pointer-based Implementation of Stacks

- **Pop** function deletes the node from the top of the stack and returns its data by reference

```
bool Stack::Pop(int& returnvalue)
{
        if (IsEmpty())
        {
                cout<<“underflow error”;
                return false;
        }
        node* tempptr = top;
        returnvalue = top->data;
        top = top->next;
        delete tempptr;
        return true;
}
```

Pointer-based Implementation of Stacks

```
//Destructor
Stack::~Stack()
{
	makeNull();
}

//makeNull() resets top pointer to NULL
void Stack::makeNull() {
	int x;
	while (Pop(x));
}
```